1、基本变量类型

布尔型
字符型
整型
浮点型
双浮点型
无类型
宽字符型

```
typedef short int wchar_t;
```

所以 wchar_t 实际上的空间是和 short int 一样。

修饰符	描述
signed	表示有符号类型（默认）
unsigned	表示无符号类型
short	表示短整型
long	表示长整型
const	表示常量，值不可修改
volatile	表示变量可能被意外修改，禁止编译器优化
mutable	表示类成员可以在 const 对象中修改

复合，加载在具体值后面

```
type:       **********size**********bool:       所占字节数：1    最大值：1       最小值：0char:       所占字节数：1    最大值：       最小值：?signed char:    所占字节数：1    最大值：       最小值：?unsigned char:
```

所占字节数：1 最大值：? 最小值：wchar_t: 所占字节数：4 最大值：2147483647 最小值：-2147483648short: 所占字节数：2 最大值：32767 最小值：-32768int: 所占字节数：4 最大值：2147483647 最小值：-2147483648unsigned: 所占字节数：4 最大值：4294967295 最小值：0long: 所占字节数：8 最大值：9223372036854775807 最小值：-9223372036854775808unsigned long: 所占字节数：8 最大值：18446744073709551615 最小值：0double: 所占字节数：8 最大值：1.79769e+308 最小值：2.22507e-308long double: 所占字节数：16 最大值：1.18973e+4932 最小值：3.3621e-4932float: 所占字节数：4 最大值：3.40282e+38 最小值：1.17549e-38size_t: 所占字节数：8 最大值：18446744073709551615 最小值：0string: 所占字节数：24

数据类型	描述	示例
数组	相同类型元素的集合	int arr[5] = {1, 2, 3, 4, 5};
指针	存储变量内存地址的类型	int* ptr = &x;
引用	变量的别名	int& ref = x;
函数	函数类型，表示函数的签名	int func(int a, int b);
结构体	用户定义的数据类型，可以包含多个不同类型的成员	struct Point { int x; int y; };
类	用户定义的数据类型，支持封装、继承和多态	class MyClass { ... };
联合体	多个成员共享同一块内存	union Data { int i; float f; };
枚举	用户定义的整数常量集合	enum Color { RED, GREEN, BLUE };

类型别名

别名	描述	例子
typedef	为现有类型定义别名	typedef int MyInt;
using	为现有类型定义别名（C++11 引入）	using MyInt = int;

数据类型	描述	示例

数据类型	描述	示例
std::string	字符串类型	std::string s = "Hello";
std::vector	动态数组	std::vector<int> v = {1, 2, 3};
std::array	固定大小数组（C++11 引入）	std::array<int, 3> a = {1, 2, 3};
std::pair	存储两个值的容器	std::pair<int, float> p(1, 2.0);
std::map	键值对容器	std::map<int, std::string> m;
std::set	唯一值集合	std::set<int> s = {1, 2, 3};

---

typedef 声明

您可以使用 **typedef** 为一个已有的类型取一个新的名字。下面是使用 typedef 定义一个新类型的语法：

```
typedef type newname;
```

例如，下面的语句会告诉编译器，feet 是 int 的另一个名称：

```
typedef int feet;
```

现在，下面的声明是完全合法的，它创建了一个整型变量 distance：

```
feet distance;
```

枚举类型

```
enum 枚举名{
      标识符[=整型常数],
      标识符[=整型常数],
...
    标识符[=整型常数]
    } 枚举变量;
```

如果枚举没有初始化，即省掉"=整型常数"时，则从第一个标识符开始。
例如，下面的代码定义了一个颜色枚举，变量 c 的类型为 color。最后，c 被赋值为 "blue"。

```
enum color { red, green, blue } c;

c = blue;
```

默认情况下，第一个名称的值为 0，第二个名称的值为 1，第三个名称的值为 2，以此类推。

（下标索引）

赋予一个特殊的值
在下面的枚举中，green 的值为 5。

```
enum color { red, green=5, blue };
```

在这里，blue 的值为 6，因为默认情况下，每个名称都会比它前面一个名称大 1，但 red 的值依然为 0。

静态转换（Static Cast）

```
int i = 10;
float f = static_cast<float>(i);
// 静态将 int 类型转换为 float 类型
```

动态转换（Dynamic Cast）

向下转换
动态转换通常用于将一个基类指针或引用转换为派生类指针或引用。
dynamic_cast<目标类型>（表达式）

特性	指针类型	引用类型
转换失败返回值	返回 nullptr	抛出 std::bad_cast 异常
适用场景	向下转换、运行时类型检查	向下转换、运行时类型检查
性能开销	较高	较高
基类要求	必须具有虚函数	必须具有虚函数

常量转换（Const Cast）

```
const int i = 10;
int& r = const_cast<int&>(i);
// 常量转换，将 const int 转换为 int
```

只改变 const（常变成变量，而不改变值和数据类型）

重新解释转换（Reinterpret Cast）

```
int i = 10;
float f = reinterpret_cast<float&>(i);
// 重新解释将 int 类型转换为 float 类
```

2、变量的声明和定义

1.

整数类型（Integer Types）：

2.

- int：用于表示整数，通常占用 4 个字节。
- short：用于表示短整数，通常占用 2 个字节。
- long：用于表示长整数，通常占用 4 个字节。
- long long：用于表示更长的整数，通常占用 8 个字节。

3.

浮点类型（Floating-Point Types）：

4.

- float：用于表示单精度浮点数，通常占用 4 个字节。
- double：用于表示双精度浮点数，通常占用 8 个字节。
- long double：用于表示更高精度的浮点数，占用字节数可以根据实现而变化。

5.

字符类型（Character Types）：

6.

- char：用于表示字符，通常占用 1 个字节。

- wchar_t：用于表示宽字符，通常占用 2 或 4 个字节。
- char16_t：用于表示 16 位 Unicode 字符，占用 2 个字节。
- char32_t：用于表示 32 位 Unicode 字符，占用 4 个字节。

7.

布尔类型（Boolean Type）：

8.

- bool：用于表示布尔值，只能取 true 或 false。

9.

枚举类型（Enumeration Types）：

10.

- enum：用于定义一组命名的整数常量。

11.

指针类型（Pointer Types）：

12.

- type*：用于表示指向类型为 type 的对象的指针。

13.

数组类型（Array Types）：

14.

- type[]或 type[size]：用于表示具有相同类型的元素组成的数组。

15.

结构体类型（Structure Types）：

16.

- struct：用于定义包含多个不同类型成员的结构。

17.

类 类型（Class Types）：

18.

- class：用于定义具有属性和方法的自定义类型。

19.

共用体类型（Union Types）：

20.

- union：用于定义一种特殊的数据类型，它可以在相同的内存位置存储不同的数据类型。

可以使用类名和作用域解析运算符 :: 来访问类中变量

3、 变量的默认初始化

整数常量

```
212 // 合法的
215u // 合法的
0xFeeL // 合法的
078 // 非法的：8 不是八进制的数字
032UU // 非法的：不能重复后缀
```

字符串

可以使用 \ 做分隔符，把一个很长的字符串常量进行分行。

定义常量

在 C++ 中，有两种简单的定义常量的方式：

- 使用 #define 预处理器。
- Define 变量= 数字
- 使用 const 关键字。

下面列出了数据类型修饰符：

- signed：表示变量可以存储负数。对于整型变量来说，signed 可以省略，因为整型变量默认为有符号类型。
-
- unsigned：表示变量不能存储负数。对于整型变量来说，unsigned 可以将变量范围扩大一倍。
-
- short：表示变量的范围比 int 更小。short int 可以缩写为 short。
-

- long：表示变量的范围比 int 更大。long int 可以缩写为 long。
- long long：表示变量的范围比 long 更大。C++11 中新增的数据类型修饰符。
- float：表示单精度浮点数。
- double：表示双精度浮点数。
- bool：表示布尔类型，只有 true 和 false 两个值。
- char：表示字符类型。
- wchar_t：表示宽字符类型，可以存储 Unicode 字符。

定义时候无符号整数无法存储负数，会转变为巨大的正数使得运算结果出现问题

C++ 中的类型限定符

类型限定符提供了变量的额外信息，用于在定义变量或函数时改变它们的默认行为的关键字。

限定符	含义
const	**const** 定义常量，表示该变量的值不能被修改。

volatile	修饰符 **volatile** 告诉该变量的值可能会被程序以外的因素改变， 如硬件或其他线程。。
restrict	由 **restrict** 修饰的指针是唯一一种访问它所指向的对象的方式。 只有 C99 增加了新的类型限定符 restrict。
mutable	mutable 用于修饰类的成员变量。被 mutable 修饰的成员变量可以被修改 ，即使它们所在的对象是 const 的。
static	用于定义静态变量，表示该变量的作用域仅限于当前文件或当前函数内， 不会被其他文件或函数访问。
register	用于定义寄存器变量，表示该变量被频繁使用，可以存储在 CPU 的寄存器中， 以提高程序的运行效率。

```
register 关键字建议编译器将变量 num 存储在寄存器中
    // 以提高程序执行速度
    // 但是实际上是否会存储在寄存器中由编译器决定
```

4、auto 类型说明符

-

auto：这是默认的存储类说明符，通常可以省略不写。auto 指定的变量具有自动存储期，即它们的生命周期仅限于定义它们的块（block）。auto 变量通常在栈上分配。

-
-

register：用于建议编译器将变量存储在 CPU 寄存器中以提高访问速度。在 C++11 及以后的版本中，register 已经是一个废弃的特性，不再具有实际作用。

-
-

static：用于定义具有静态存储期的变量或函数，它们的生命周期贯穿整个程序的运行期。在函数内部，static 变量的值在函数调用之间保持不变。在文件内部或全局作用域，static 变量具有内部链接，只能在定义它们的文件中访问。

-

-

extern：用于声明具有外部链接的变量或函数，它们可以在多个文件之间共享。默认情况下，全局变量和函数具有 extern 存储类。在一个文件中使用 extern 声明另一个文件中定义的全局变量或函数，可以实现跨文件共享。

-

-

mutable (C++11)：用于修饰类中的成员变量，允许在 const 成员函数中修改这些变量的值。通常用于缓存或计数器等需要在 const 上下文中修改的数据。

-

-

thread_local (C++11)：用于定义具有线程局部存储期的变量，每个线程都有自己的独立副本。线程局部变量的生命周期与线程的生命周期相同。

-

从 C++ 17 开始，auto 关键字不再是 C++ 存储类说明符，且 register 关键字被弃用

auto 关键字用于两种情况：
声明变量时根据初始化表达式自动推断该变量的类型、
声明函数时函数返回值的占位符。

```
auto x1 = 5, x2 = 5.0, x3='r';//错误，必须是初始化为同一类型
```

5、循环 for、while、分支语句，条件语句

6、Cpp 类型转换（隐式类型转换、static_cast 显式类型转换）

隐式类型转换：

没有写语法编译器自动帮你转换

1. 会在不同算术计算类型中把他们转化为更高级的类型

顺序为：int -- unsigned int------long----unsigned long-----long long-----unsigned long long

2、小于 int 的类型会被升为 int

Char a=10;

Char b=10

Int c=a+b

(先转 int 再相加)

3、赋值

E. g.:e

Int d=1.b

会自动变为整数 1（往类型上变化）

4、函数传参

实参和形参不一样 ing 会转换

Be like

Void f(double x);

F(10);

//10 自动从 int 转换为 double

static_cast 显式类型转换

Static 类型

用来保持全局变量的存在

static 修饰符也可以应用于全局变量。

当 static 修饰全局变量时，会使变量的作用域限制在声明它的文件内。

```
void func(void);
static int count = 10; /* 全局变量 */
int main() { while(count--) { func(); } return 0; }
 // 函数定义 void func( void )
 { static int i = 5;
 // 局部静态变量
 i++;
 std::cout << "变量 i 为 " << i ; std::cout << ", 变量 count 为 " << count << std::endl; }
```

Cast 类型

7、c++运算符（算术、逻辑与关系、递增递减、条件、成员访问）

8、数组、指针和引用

9、函数（声明和定义、形参和实参、传引用函数、函数重载、内联函数）

10、类（定义、成员函数和成员变量的声明和定义、this 指针、构造函数、构造函数和初始

化列表、常量成员和常量成员变量、访问控制（public、private）、类的作用域、继承和多态（基类派生类的定义、派生类别的类型转换、动态绑定、虚函数、纯虚函数与抽象函数））
11、Vector 创建与简单操作、vector 遍历的几种方式

注释

1、 /*内容*/（不可以嵌套）
2、 //
3、
#if 0
…… ……
#endif
4、
#if condition
内容
#else
内容
#endif

上学迟到

精确小数值

#include <iomanip>

Cout<<fixed<<setprecision(数字)<<endl;

精确补 0

#include<iomanip>

…… ……

Cout<<setw(宽度)<<setfill(‘0’)<<你的变量

Cin>>a>>b;

在输入的时候已经表示 a b(此时有空格)

浮点数取余：使用 fmod 函数

```
#include <cmath>
```

……

变量=fmod(被除数，除数)

向下取整

floor()

向上取整

ceil()

四舍五入：

round（）

浮点数取绝对值：

fabs( )

“”为字符串类型：string

‘’为 char 类型：字符

using 函数的使用方法

Using ull =unsigned long long;//先起别名,相当于把后面的变量改为 ull

Ull x=100;

Cout<<x<<endl;

X++和 ++a 的区别：

int x = 5;

cout << x++ << endl; // 5 先输出 5，再 x 变成 6

cout << x << endl; // 6

cout << ++x << endl; // 7 先 x 变成 7，再输出 7

cout << x << endl; // 7

你错在哪：

· x++：先用后加，输出 5，然后 x 变 6

· ++x：先加后用，x 变 7，输出 7

· 最后一个 x 已经是 7 了，你写 6，错。

口诀：

· x++：先交值，再加

· ++x：先加，再交值

指针、引用：

数组：

Int arr[3]={10,20,30}

用数组名字【下标】　　来索引

遍历：

For(int i =0;i<3;i++)

{

　　Cout<<arr[i]<<endl;

}

数组不检查越界，访问arr[100]编译能过，运行时可能会崩溃

指针：

定义：存地址的变量

Int a=10;

Int *p=&a;　　//取地址，p存a的地址

Cout<<*p;　　//解引用，输出10

*p=20;　　//通过指针改a，a变成20

&a:变量的地址

*p解引用，取地址里面的值

（*解引用，&表示存地址）

使用指针的目的：

直接操作内存；

函数之间传递大数据时候避免复刻

动态分配内存（new）

引用

变量的别名，操作别名相当于操作变量

Int a=10;

Int&r=a；　声明引用，是a的别名

r =30　　改r相当于改a

Cout<<a;

引用特点：

必须初始化，int &r;不合法

一旦绑定，不能改别的变量

没有“空引用”

引用传参（tip）（重点）：

普通传参：改变不了原变量

```
Void add (int x)
{
X+=10;    //改的是副本，外面 a 不变
}
Int main()
{
Int a=5;
add(a);
Cout<<a;                    //5 没改变
}
```

引用传参

能改变原变量：

```
Void add(int&x):        //加上&才是引用
    X+=10;
    add(a);
```

cout<<a;

关键区别：

声明的时候加& ： void add(int &x)

调用时候不加&: add(a)

指针和引用的区别：

	指针	引用
写法	Int *p=&a;	Int &r=a;
能否为空	可以 int *p=nullptr;	不能为空
能否改指向	可以 p=&b;	绑定后不能改
取原值	*p	r
传参	Void f(int *p)	Void f(int &r)

考法：

给代码写输出：

int a=10;

Int *p=&a;

*p=20;

Cout<<a;

判断地址还是值

```
Int a=10;
Int*p=&a;
Cout<<p<<endl;                    //传出地址
Cout<<*p<<endl;                   //传出值
```

引用传参改变变量;

```
Void swap(int&a,int&b)
{
Int t=a;
A=b;
B=t;
}
```

问题:

改了p指针的指向的时候，指向的值会改变吗:

不会，分两种情况

1、换目标:

```
int a=10,b=20;
Int *p=&a;   (一开始指向a)
```

P=&b;　　（后面指向 b）

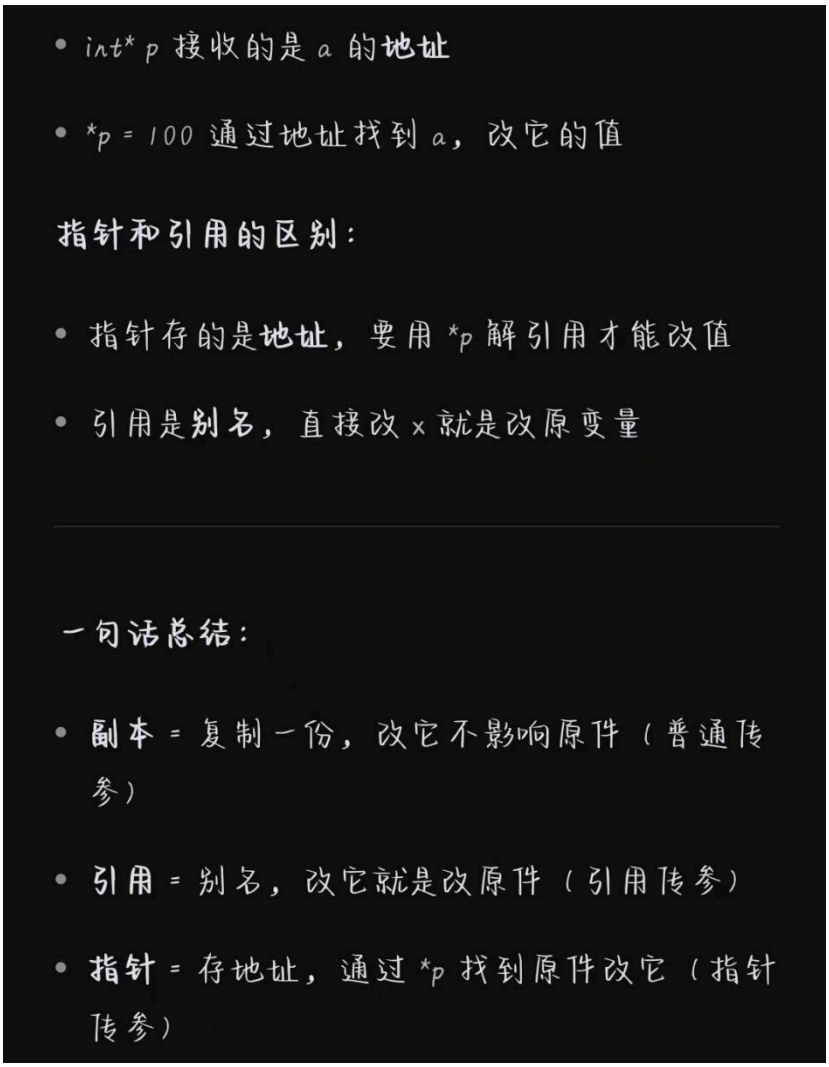
//此时 a 还是 10，b 还是 20，只是让 p 换了目标，原来的值没有改变

2、改目标内容

Int a=10;

Int *p=&a;

*p=30;　　//这个时候改了目标指向的那个值



Swap()是标准库函数

#include<algorithm>

Swap(a,b);

(int x)　指的是普通传参（传副本）

（int &x）　指的是引用传参（内外函数一起改变）

（int*p）　指的是指针传参（传地址）

<int>(变量)　（类型转换）把……转换为int类型

<int>

表示这个容器里面装的是int类型，模板参数

Sizeof

作用查看类型/变量大小：（返回值为：size_t）

Sizeof(int)

Double d;

Sizeof(d)

特点：

编译时运行不长生特殊开销

返回值是 size_t(无符号整数)

Sizeof(数组名)得到的是整个数组的字节数，但数组作为函数参数时会退化为指针，此时 sizeof 得到的是指针的大小（比如 8 字节）

对表达式使用时不会真的计算该表达式，只是推断类型，例如 sizeof(i++)不会让 i 自增

计算数组长度：

int arr[10];

Int n=sizeof/sizeof(arr[0]);

分配的是堆内存，且不初始化

Void *malloc(size_t size):

在堆上分配 size 字节

返回 void*指向内存首地址

失败返回 NULL

内容是不确定的（不像 calloc 会清零）

返回 void *（*意为指针类型的一部分，表示一个指向 void 的指针，也就是无类型指针/通用指针）

()这个可以隐式转化为任何指针类型，所以

Int *p=(int*)malloc(……)　　//cpp 必须强转

和 free 配对的



p 只存第一个元素的地址，不是整个数组的地址。

数组在内存里是这样排的：

cpp 复制

```cpp
int arr[3] = {10, 20, 30};
```

p++ 不是简单加 1，而是加一个 int 的大小（4 字节），指向下一个元素。

为什么 arr 代表首地址，不是整个数组？

这是 C++ 的设计：

- 数组名在大多数情况下会“退化”成**指向首元素的指针**
- 所以 int* p = arr; 合法，arr 被当成 &arr[0]
- 但 arr 本身不是一个指针变量，它只是首地址的“代表”

- 数组名 arr 本身就代表首元素地址，所以可以 int* p = arr;

还有一点容易混的：

```cpp
int* p = arr;  //p 是 int* 类型，指向首元素
```

如果你想“指向整个数组”，那要用数组指针：

```cpp
int (*p)[3] = &arr;  //p 指向整个数组，+1 会跳过整个数组
```

但这种写法考纲外，不用管。 你只要记住普通的 int* p = arr; 指向首元素就行。



Void：

当返回值是 void 类型的时候，没有返回值

所以不能把 void 定义的函数在后续 main 函数中进行赋值 int 类型

即不能用：int 变量=void 定义函数（变量）

就直接写函数（a）

在 cout 输出就行